

CGI Web App with FOSS

Apply the concept of Common Gateway Interface to develop a simple web-based application and explain its working with an example. in detail in Free and Open Source Software Tools

Here is a **detailed answer** for your assignment:

Apply the Concept of Common Gateway Interface to Develop a Simple Web-Based Application (Using Free and Open Source Software Tools)

1. Introduction to Common Gateway Interface (CGI)

The **Common Gateway Interface (CGI)** is a standard protocol used to connect a web server with external programs (such as Python, Perl, or Shell scripts). CGI allows web servers to generate dynamic web pages based on user input.

In CGI:

- The user fills a form on a web page.
- The web server sends the form data to a CGI program.
- The CGI program processes the data.
- The CGI program sends the result back to the web server.
- The web server displays the result in the browser.

CGI programs are usually written in **Python, Perl, C, or Bash**, which are **Free and Open Source Software (FOSS)** tools.

2. Free and Open Source Software Tools Used

Tool	Purpose
Apache HTTP Server	Web server
Python	CGI scripting

Tool	Purpose
HTML	Web page design
Linux	Platform to run server
XAMPP	Local server (Apache included)

All the above tools are **free and open source**.

3. Architecture of CGI

```
graph TD
    User[User (Browser)] --> WS1[Web Server (Apache)]
    WS1 --> CGI[CGI Program (Python Script)]
    CGI --> WS2[Web Server]
    WS2 --> User2[User Browser (Output Web Page)]
```

4. Example Application – Student Form Using CGI (Python)

Step 1 – Create HTML Form (form.html)

</> HTML

```
<html>
<body>
<h2>Student Form</h2>
<form action="/cgi-bin/student.py" method="post">
Name: <input type="text" name="name"><br><br>
Age: <input type="text" name="age"><br><br>
<input type="submit" value="Submit">
</form>
```

```
</body>
</html>
```

Step 2 – Create CGI Script (student.py)

</> Python

```
#!/usr/bin/python3

import cgi
import cgitb
cgitb.enable()

print("Content-type:text/html\r\n\r\n")

form = cgi.FieldStorage()

name = form.getvalue("name")
age = form.getvalue("age")

print("<html>")
print("<body>")
print("<h2>Student Details</h2>")
print("<p>Name:", name, "</p>")
print("<p>Age:", age, "</p>")
print("</body>")
print("</html>")
```

Step 3 – Place Files in Server

- Place **form.html** in `htdocs`
- Place **student.py** in `cgi-bin`
- Give permission:

```
chmod +x student.py
```

Step 4 – Run Application

Open browser:

```
http://localhost/form.html
```

Enter name and age → Click Submit → Output will be displayed.

5. Working of CGI (Step-by-Step Explanation)

1. User opens **form.html** in browser.
 2. User enters data and clicks submit.
 3. The browser sends data to the web server.
 4. The web server runs the CGI script (**student.py**).
 5. The CGI script reads form data using `FieldStorage()`.
 6. The CGI script processes the data.
 7. The CGI script sends HTML output.
 8. The web server sends output to the browser.
 9. The browser displays the result.
-

6. Advantages of CGI

- Simple to implement
 - Language independent (Python, Perl, C, etc.)
 - Works with any web server
 - Free and open source tools available
 - Used for dynamic web pages
-

7. Disadvantages of CGI

- Slower because a new process is created for each request
 - Not suitable for large applications
 - Security risks if not configured properly
-

8. Conclusion

The **Common Gateway Interface (CGI)** is used to create dynamic web applications by connecting web servers with external programs. Using free and open source tools like Apache, Python, HTML, and Linux, we can develop simple web-based applications such as student forms, login pages, feedback forms, etc. CGI helps in processing user input and generating dynamic output web pages.

9. Viva Questions (Important)

Q1: What is CGI?

CGI is a standard protocol used to connect web server and external programs to generate dynamic web content.

Q2: Give examples of CGI languages.

Python, Perl, C, Shell Script.

Q3: What is the use of `cgi.FieldStorage()`?

It is used to read form data sent by the browser.

Q4: Name some FOSS tools used in CGI.

Apache, Linux, Python, HTML.

If you want, I can also give **CGI architecture diagram** for drawing in exam.

Apply various error handling techniques in Perl to manage runtime errors with suitable examples. in detail in Free and Open Source Software Tools

Here is a **detailed answer** for your assignment:

Apply Various Error Handling Techniques in Perl to Manage Runtime Errors (Using Free and Open Source Software Tools)

1. Introduction

Perl is a free and open source programming language widely used for scripting, system administration, CGI programming, and text processing. While running Perl programs, **runtime errors** may occur due to file errors, user input errors, database errors, etc. To prevent program crashes, Perl provides various **error handling techniques**.

Error handling helps to:

- Prevent program termination
- Display proper error messages
- Handle unexpected situations

- Improve program reliability

2. Types of Errors in Perl

Error Type	Description
Syntax Error	Error in program syntax
Runtime Error	Error during program execution
Logical Error	Program runs but gives wrong output

This assignment focuses on **runtime errors**.

3. Error Handling Techniques in Perl

3.1 Using `die` Function

The `die` function is used to display an error message and terminate the program when a runtime error occurs.

Example: File Open Error

```
</> Perl
#!/usr/bin/perl
use strict;
use warnings;

my $file = "data.txt";

open(my $fh, "<", $file) or die "Cannot open file: $!";

print "File opened successfully\n";
```

Explanation:

- If the file does not exist, the program stops.
 - `$!` displays the system error message.
-

3.2 Using `warn` Function

The `warn` function displays a warning message but **does not stop** the program.

Example:

```
</> Perl
#!/usr/bin/perl
use strict;
use warnings;

my $num = 0;

if ($num == 0) {
    warn "Number is zero, division not possible\n";
} else {
    print 10/$num;
}

print "Program continues...\n";
```

3.3 Using `eval` Block (Exception Handling)

The `eval` block is used to catch runtime errors without stopping the program.

Example: Division Error Handling

```
</> Perl
#!/usr/bin/perl
use strict;
use warnings;

eval {
    my $a = 10;
    my $b = 0;
    my $c = $a / $b;
    print "Result: $c\n";
};

if ($@) {
    print "Error occurred: $@\n";
}
```

Explanation:

- `eval` executes code.

- If error occurs, it is stored in `$@`.
 - Program continues execution.
-

3.4 Using `or` Operator for Error Handling

Perl commonly uses `or` for error handling with file operations.

Example:

```
<> Perl
#!/usr/bin/perl
use strict;
use warnings;

open(FILE, "data.txt") or print "File not found\n";
```

3.5 Using `Try::Catch` Module (FOSS Module)

`Try::Catch` is a free and open source Perl module used for exception handling similar to Java.

Example:

```
<> Perl
#!/usr/bin/perl
use strict;
use warnings;
use Try::Catch;

try {
    die "Runtime error occurred!";
}
catch {
    print "Caught Error: $_";
};
```

4. Summary Table

Method	Description
die	Stops program with error message
warn	Displays warning and continues
eval	Handles exception
or	Used with file handling
Try::Catch	Advanced exception handling

5. Real-Time Example – File Handling with Error Handling

</> Perl

```
#!/usr/bin/perl
use strict;
use warnings;

my $filename = "data.txt";

unless(open(FILE, $filename)) {
    die "Cannot open file: $!";
}

while(my $line = <FILE>) {
    print $line;
}

close(FILE) or warn "Cannot close file: $!";
```

6. Advantages of Error Handling in Perl

- Prevents sudden program crash
- Helps debugging
- Improves program stability
- Provides user-friendly error messages
- Useful in CGI and system scripts

7. Conclusion

Perl provides powerful error handling techniques such as **die**, **warn**, **eval**, or **operator**, and **Try::Catch module** to manage runtime errors. These techniques help developers build reliable and stable applications using **free and open source software tools** like Perl. Proper error handling ensures smooth execution even when unexpected runtime errors occur.

8. Viva Questions

Q1: What is runtime error?

Error that occurs during program execution.

Q2: What is `$@` in Perl?

It stores error message in eval block.

Q3: Difference between die and warn?

die stops program, warn continues program.

Q4: Which module is used for exception handling in Perl?

Try::Catch module.

If you want, I can also give **short 2-page exam answer format**.

give me the output for the above programs in error handling

Here are the **outputs for the Perl error handling programs** you wrote above.

Outputs for Perl Error Handling Programs

1. Output for `die` Function Program

Program: File Open Error using `die`

If the file **data.txt** does **NOT** exist, output will be:

```
Cannot open file: No such file or directory at student.pl line 6.
```

If the file **exists**, output will be:

```
File opened successfully
```

Explanation:

`die` stops the program immediately when an error occurs.

2. Output for `warn` Function Program

Program: Division Warning

Since `$num = 0`, output will be:

```
Number is zero, division not possible  
Program continues...
```

Explanation:

`warn` prints a warning but the program continues execution.

3. Output for `eval` Block Program

Program: Division Error Handling using `eval`

Since division by zero occurs, output will be:

```
Error occurred: Illegal division by zero at student.pl line 8.
```

Explanation:

- Error is caught by `eval`
 - Error stored in `$@`
 - Program does not crash
-

4. Output for `or` Operator Program

Program: File Open using `or`

If **data.txt** does NOT exist, output:

```
File not found
```

If file **exists**, output:

```
(No output, program runs normally)
```

5. Output for `Try::Catch` Module Program

Program: Exception Handling

Output:

```
Caught Error: Runtime error occurred!
```

6. Output for Real-Time File Handling Program

Program: File Handling with `die` and `warn`

Case 1 – File does NOT exist

```
Cannot open file: No such file or directory at student.pl line 7.
```

Case 2 – File exists

If **data.txt** contains:

```
Hello  
Welcome  
Perl Programming
```

Output will be:

```
Hello  
Welcome  
Perl Programming
```

If file closing fails:

Cannot close file: <error message>

Summary Table

Method	Output When Error Occurs
die	Program stops with error message
warn	Warning message, program continues
eval	Error caught and printed
or	Prints custom error message
Try::Catch	Catches and prints error

Small Viva Point

Special Variables in Perl Error Handling:

Variable	Meaning
<code>\$!</code>	System error message
<code>\$@</code>	Eval error message
<code>\$_</code>	Default variable in catch

If you want, I can also give **flow diagram of Perl error handling** for exam.

Illustrate the role of bug tracking system and package managers in open software development. Compare at least two widely used bug tracking system. in Free and Open Source Software Tools

Here is a detailed assignment answer for:

Illustrate the Role of Bug Tracking System and Package Managers in Open Software Development. Compare at least Two Widely Used Bug Tracking Systems (FOSS Tools)

1. Introduction

In **Open Source Software Development**, many developers work together from different locations. To manage software development properly, two important tools are used:

- **Bug Tracking System**
- **Package Manager**

These tools are part of **Free and Open Source Software (FOSS)** development because they help manage code, bugs, updates, and software installation.

2. Role of Bug Tracking System in Open Software Development

A **Bug Tracking System** is software used to:

- Report bugs (errors)
- Track bug status
- Assign bugs to developers
- Manage bug fixing process
- Maintain bug history

Functions of Bug Tracking System

1. Bug reporting
2. Bug tracking
3. Assigning bugs to developers
4. Setting priority
5. Tracking status (Open, In Progress, Fixed, Closed)
6. Communication between team members

Examples of Bug Tracking Systems

- Bugzilla
- Jira
- Redmine

- [GitHub Issues](#)

Bug Life Cycle

Bug Reported → Bug Confirmed → Assigned → In Progress → Fixed → Tested → Closed

Advantages

- Organized bug management
- Improves software quality
- Helps teamwork
- Maintains bug history
- Improves communication

3. Role of Package Manager in Open Software Development

A **Package Manager** is used to:

- Install software
- Update software
- Remove software
- Manage software dependencies
- Download packages from repositories

Examples of Package Managers

- [APT](#) (Ubuntu/Debian)
- [YUM](#) (RedHat/CentOS)
- [DNF](#)
- [npm](#)
- [pip](#)

Functions of Package Manager

Function	Description
Install	Install new software
Update	Update software
Remove	Remove software

Function	Description
Dependency Management	Install required libraries
Search	Search software packages

Example Commands

Package Manager	Command
APT	<code>sudo apt install python3</code>
YUM	<code>sudo yum install httpd</code>
pip	<code>pip install flask</code>
npm	<code>npm install express</code>

Advantages

- Easy installation
- Automatic dependency handling
- Easy updates
- Saves time
- Maintains software versions

4. Comparison of Two Bug Tracking Systems

Comparison: Bugzilla vs Jira

Feature	Bugzilla	Jira
Developed By	Mozilla	Atlassian
Type	Open Source	Commercial
Platform	Web-based	Web-based

Feature	Bugzilla	Jira
Customization	High	High
Ease of Use	Medium	Easy
Cost	Free	Paid
Best For	Open source projects	Enterprise projects

Explanation

Bugzilla

- Open source bug tracking tool
- Developed by Mozilla
- Used in open source projects
- Tracks bugs and issues
- Free to use

Jira

- Developed by Atlassian
- Used for bug tracking and project management
- Supports Agile (Scrum, Kanban)
- Paid software but widely used

5. Difference Between Bug Tracking System and Package Manager

Bug Tracking System	Package Manager
Used to track bugs	Used to install software
Used by developers	Used by users/admin
Manages issues	Manages software packages
Example: Bugzilla	Example: APT

6. Conclusion

Bug tracking systems and package managers play an important role in **open source software development**. Bug tracking systems like Bugzilla and Jira help developers track and fix bugs, while package managers like APT, YUM, pip, and npm help install and manage software packages and dependencies. These tools improve software quality, development speed, and system management in free and open source software development.

7. Viva Questions

Q1: What is bug tracking system?

Software used to track and manage bugs.

Q2: What is package manager?

Tool used to install, update, and manage software packages.

Q3: Name two bug tracking tools.

Bugzilla, Jira.

Q4: Name two package managers.

APT, YUM.

Q5: Who developed Bugzilla?

Mozilla.

If you want, I can also give **diagram for bug tracking system and package manager architecture** for exam drawing.

Demonstrate various features of Samba with examples in Free and Open Source Software Tools in detail

Here is a **detailed assignment answer**:

Demonstrate Various Features of Samba with Examples (Free and Open Source Software Tools)

1. Introduction to Samba

Samba is a **Free and Open Source Software (FOSS)** that allows file and printer sharing between **Linux/Unix** and **Windows** systems. Samba uses the **SMB (Server Message Block)** protocol to communicate with Windows systems.

This means:

- Linux can share files with Windows
 - Linux can access Windows shared folders
 - Linux can act as a file server
 - Linux can act as a print server
 - Linux can act as a Domain Controller
-

2. Features of Samba

2.1 File Sharing

Samba allows sharing files between Linux and Windows systems.

Example Configuration

Samba configuration file:

```
/etc/samba/smb.conf
```

Example:

```
<> INI
[SharedFolder]
path = /home/user/shared
browseable = yes
read only = no
guest ok = yes
```

Restart Samba:

```
sudo systemctl restart smbd
```

Now Windows can access:

```
\\192.168.1.10\SharedFolder
```

Feature: Cross-platform file sharing.

2.2 Printer Sharing

Samba allows Linux printer to be shared with Windows.

Example:

```
</> INI
[Printer]
path = /var/spool/samba
printable = yes
guest ok = yes
read only = yes
```

Feature: Printer sharing across network.

2.3 User Authentication

Samba provides **user-based security**.

Create Samba user:

```
sudo smbpasswd -a user1
```

Secure configuration:

```
</> INI
[SecureShare]
path = /home/user/secure
valid users = user1
read only = no
```

Feature: Only authorized users can access files.

2.4 Samba as Domain Controller

Samba can act as a **Primary Domain Controller (PDC)** like Windows Server.

Features:

- Centralized login
- User management
- Group management
- Policy management

Feature: Network user management.

2.5 Backup Server

Samba can be used as a **backup storage server** where all client systems store backup files.

Example:

```
</> INI
[Backup]
path = /backup
read only = no
browseable = yes
```

2.6 Network Browsing

Samba allows Windows systems to see Linux machine in **Network Neighborhood**.

Command to check:

```
smbclient -L localhost
```

2.7 Access Control

Samba allows permission control:

- Read only
- Read and write
- Guest access
- User access

Example:

```
</> INI
[Project]
path = /project
```

```
read only = no
valid users = user1, user2
```

3. Summary of Samba Features

Feature	Description
File Sharing	Share files between Linux and Windows
Printer Sharing	Share printers
Authentication	User login security
Domain Controller	Centralized network control
Backup Server	Store backup files
Network Browsing	Visible in Windows network
Access Control	Permission-based access

4. Advantages of Samba

- Free and Open Source
 - Works with Windows and Linux
 - Easy file sharing
 - Secure
 - Centralized management
 - Low cost (no Windows Server needed)
-

5. Disadvantages of Samba

- Configuration is complex
- Performance slower than Windows Server in some cases
- Requires Linux knowledge

6. Real-Time Example Scenario

Office Network Example:

- One Linux server with Samba installed
- 10 Windows client computers
- Shared folder for employees
- Shared printer
- User login authentication

Result:

- Employees access shared files
 - Employees use shared printer
 - Admin controls permissions
-

7. Conclusion

Samba is a powerful **Free and Open Source Software tool** used for file sharing, printer sharing, authentication, and domain control between Linux and Windows systems. It is widely used in organizations to create low-cost file servers and network storage systems.

8. Viva Questions

Q1: What is Samba?

Samba is an open source software used for file and printer sharing between Linux and Windows.

Q2: Which protocol Samba uses?

SMB (Server Message Block).

Q3: Configuration file of Samba?

```
smb.conf
```

Q4: Command to add Samba user?

```
smbpasswd -a username
```

Q5: Samba runs as which service?

```
smbd and nmbd
```

If you want, I can also give **Samba architecture diagram** for exam drawing.